Member:

- Naphat Utabuawong 67070501015
- Thanatip Nitinantakul 67070501023
- Benjamin Garforth 67070501031

Problem 1

Given the following initial value: R1 = 5, R2 = 10, R3 = 0, and MEM[100] = 20 with the following instruction Sequence.

```
1. LOAD R3, [100]
2. ADD  R1, R3, R2
3. SUB  R2, R1, R3
4. STORE R2, [104]
```

Fill in the value of each register and memory location.

Ri/LOC	R1	R2	R3	[100]	[104]
Initial	5	10	0	20	x
Inst 1.	5	10	20	20	x
Inst 2.	30	10	20	20	x
Inst 3.	30	10	20	20	x
Inst 4.	30	10	20	20	10

Problem 2

1. Write a program to calculate the 44th Fibonacci number without using any special library in C language.

Code:

```
#include <stdio.h>

int main(){
    int n = 44;
    int prev1 = 1;
    int prev2 = 0;
    int curr = 0;

    for (int i = 1; i < n; i++) {
        curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }

    printf("%d ", curr);
    return 0;
}
```

2. Convert your C code into ARM assembly.

Assembly:

```
.LC0:
     .string   "%d "
main:
     stp  x29, x30, [sp, -48]!
     mov  x29, sp
     mov  w0, 44
     str  w0, [sp, 28]
     mov  w0, 1
     str  w0, [sp, 44]
     str  wzr, [sp, 40]
     str  wzr, [sp, 36]
     mov  w0, 1
     str  w0, [sp, 32]
     b    .L2
.L3:
     ldr  w1, [sp, 44]
     ldr  w0, [sp, 40]
     add  w0, w1, w0
     str  w0, [sp, 36]
     ldr  w0, [sp, 44]
     str  w0, [sp, 40]
     ldr  w0, [sp, 36]
     str  w0, [sp, 44]
     ldr  w0, [sp, 32]
     add  w0, w0, 1
     str  w0, [sp, 32]
.L2:
     ldr  w1, [sp, 32]
     ldr  w0, [sp, 28]
     cmp  w1, w0
     blt  .L3
     ldr  w1, [sp, 36]
     adrp x0, .LC0
     add  x0, x0, :lo12:.LC0
     bl   printf
     mov  w0, 0
     ldp  x29, x30, [sp], 48
     ret
```

- Brief explanation of how your code calculate 44th Fibonacci number.

Answer:

This program calculates the 44th Fibonacci number by iteratively building the Fibonacci sequence. The variable n specifies the position of the desired Fibonacci number. The variables prev2 and prev1 are initialized to 0 and 1, representing the first two values in the sequence. These values are used to generate the next number in each step.

Inside the loop, the next Fibonacci number is calculated by adding the two previous values and storing the result in curr. The variables are then updated so that prev2 and prev1 always hold the most recent two numbers in the sequence. This process continues until the loop reaches the 44th position. After the loop completes, curr contains the 44th Fibonacci number, which is printed as the final output.

- maps the C instruction to the assembly instruction.

Answer:

C	ASM	Explanation
int main() {	main: stp x29, x30, [sp, -48]! mov x29, sp	x29 ทำหน้าที่เป็น frame pointer ใช้ชี้ตำแหน่งอ้างอิงของ stack frame ภายในฟังก์ชัน x30 ทำหน้าที่เป็น link register ใช้เก็บที่อยู่ของคำสั่งถัดไปหลังจากฟังก์ชันจบการทำงาน (return address) คำสั่ง stp ใช้บันทึกค่าในregister x29 และ x30 ลงบน stack พร้อมทั้งจองพื้นที่ขนาด 48 ไบต์สำหรับตัวแปรภายในฟังก์ชัน คำสั่ง mov x29, sp ใช้กำหนดค่า stack pointer ปัจจุบันให้เป็นจุดอ้างอิงของ stack frame ของฟังก์ชัน
int n = 44;	mov w0, 44 str w0, [sp, 28]	- w0 = 32-bit register (because int) - ใส่ค่าคงที่ 44 ลงในregister w0 - เก็บค่า 44 ลงหน่วยความจำที่ตำแหน่ง [sp, 28] - ตำแหน่งนี้ถูกใช้แทนตัวแปร n
int prev1 = 1;	mov w0, 1 str w0, [sp, 44]	- ใส่ค่าคงที่ 1 ลงในregister w0 - เก็บค่า 44 ลงหน่วยความจำที่ตำแหน่ง [sp, 44] - ตำแหน่งนี้เก็บค่าของตัวแปร prev1
int prev2 = 0;	str wzr, [sp, 40]	- wzr เป็นregisterพิเศษที่มีค่าเป็น 0 ตลอดเวลา ฉะนั้นไม่ต้องใส่ค่าคงที่อะไรใน register w0 - เก็บค่า 0 ลงหน่วยความจำที่ตำแหน่ง [sp, 40] - ตำแหน่งนี้ถูกใช้แทนตัวแปร prev2
int curr = 0;	str wzr, [sp, 36]	- wzr เป็นregisterพิเศษที่มีค่าเป็น 0 ตลอดเวลา ฉะนั้นไม่ต้องใส่ค่าคงที่อะไรใน register w0 - เก็บค่า 0 ลงหน่วยความจำที่ตำแหน่ง [sp, 36] - ตำแหน่งนี้ถูกใช้แทนตัวแปร curr

C	ASM	Explanation
for (int i = 1; i < n; i++) {	main: mov w0, 1 str w0, [sp, 32] b .L2	- ใส่ค่าคงที่ 1 ลงใน register w0 - เก็บค่า 1 ลงหน่วยความจำที่ ตำแหน่ง [sp, 32] - ตำแหน่งนี้ถูกใช้แทนตัวแปร i

	.L3:
	ldr w0, [sp, 32]
	add w0, w0, 1
	str w0, [sp, 32]
		- อ่านค่าของ i จากหน่วยความจำที่ ตำแหน่ง [sp, 32] เข้า register w0
		- บวกค่า i ด้วย 1 แล้วนำไปเก็บค่าไว้ที่ w0
		- เก็บค่า i ลงหน่วยความจำที่ ตำแหน่ง [sp, 32]

	.L2:
	ldr w1, [sp, 32]
	ldr w0, [sp, 28]
	cmp w1, w0
	blt .L3
		- อ่านค่าของ i จากหน่วยความจำที่ ตำแหน่ง [sp, 32] เข้า register w0
		- อ่านค่าของ n จากหน่วยความจำที่ ตำแหน่ง [sp, 28] เข้า register w1
		- เปรียบเทียบค่าใน register w1 และ w0
		- blt คือ Branch if less than คือการที่ ถ้า i < n jump ไปที่ .L3 ก็คือ กลับไปทำงานใน Loop

curr = prev1 + prev2;	.L3:
	ldr w1, [sp, 44]
	ldr w0, [sp, 40]
	add w0, w1, w0
	str w0, [sp, 36]
		- อ่านค่าของ prev1 จากหน่วยความจำที่ตำแหน่ง [sp, 44] เข้า register w1
		- อ่านค่าของ prev2 จากหน่วยความจำที่ตำแหน่ง [sp, 40] เข้า register w0
		- บวกค่า จาก w1[prev1] และ w0[prev2] เข้าด้วยกัน แล้วไปเก็บค่าใน register w0
		- เก็บค่า ผลลัพธ์การบวก มาลงหน่วยความจำที่ตำแหน่ง [sp, 36]
		- ตำแหน่งนี้ถูกใช้แทนตัวแปร curr

prev2 = prev1;	ldr w0, [sp, 44] str w0, [sp, 40]	- อ่านค่าของ prev1 จากหน่วยความจำที่ตำแหน่ง [sp, 44] เข้า register w0 - เก็บค่าที่อ่านได้จาก prev1 มาลงหน่วยความจำที่ตำแหน่ง [sp, 40] - ตำแหน่งนี้ถูกใช้แทนตัวแปร prev2
prev1 = curr; }	ldr w0, [sp, 36] str w0, [sp, 44]	- อ่านค่าของ curr จากหน่วยความจำที่ ตำแหน่ง [sp, 36] เข้า register w0 - เก็บค่าที่อ่านได้จาก curr มาลงหน่วยความจำที่ตำแหน่ง [sp, 44] - ตำแหน่งนี้ถูกใช้แทนตัวแปร prev1

C	ASM	Explanation

printf("%d ", curr);	ldr w1, [sp, 36]
	adrp x0, .LC0
	add x0, x0, :lo12:.LC0
	bl printf
		- อ่านค่าของ curr จากหน่วยความจำที่ตำแหน่ง [sp, 36] เข้า register w1
		- คำนวณตำแหน่งหน่วยความจำของ format string
		- bl คือ Branch with Link คือการ Jump ไปที่การทำงาน printf

return 0; }	mov w0, 0 Ldp x29, x30, [sp], 48 ret	- ใส่ค่าคงที่ 0 ลงใน register w0 - ldp คือ Load Pair ใช้อ่านข้อมูลจากหน่วยความจำครั้งเดียว แล้วนำไปใส่ใน register 2 ตัวพร้อมกัน - อ่านค่า frame pointer (x29) และ return address (x30) กลับจาก stack พร้อมทั้งเพิ่มค่า stack pointer กลับ 48 ไบต์ เพื่อคืนพื้นที่ stack ที่ถูกจองไว้ - กระโดดกลับไปยังตำแหน่งที่เก็บไว้ใน x30 เพื่อกลับไปทำงานต่อในฟังก์ชันที่เรียก main

- Include the command(s)/steps used to convert C into ARM assembly.
Answer:

Step 1: Install the Arm GNU Toolchain

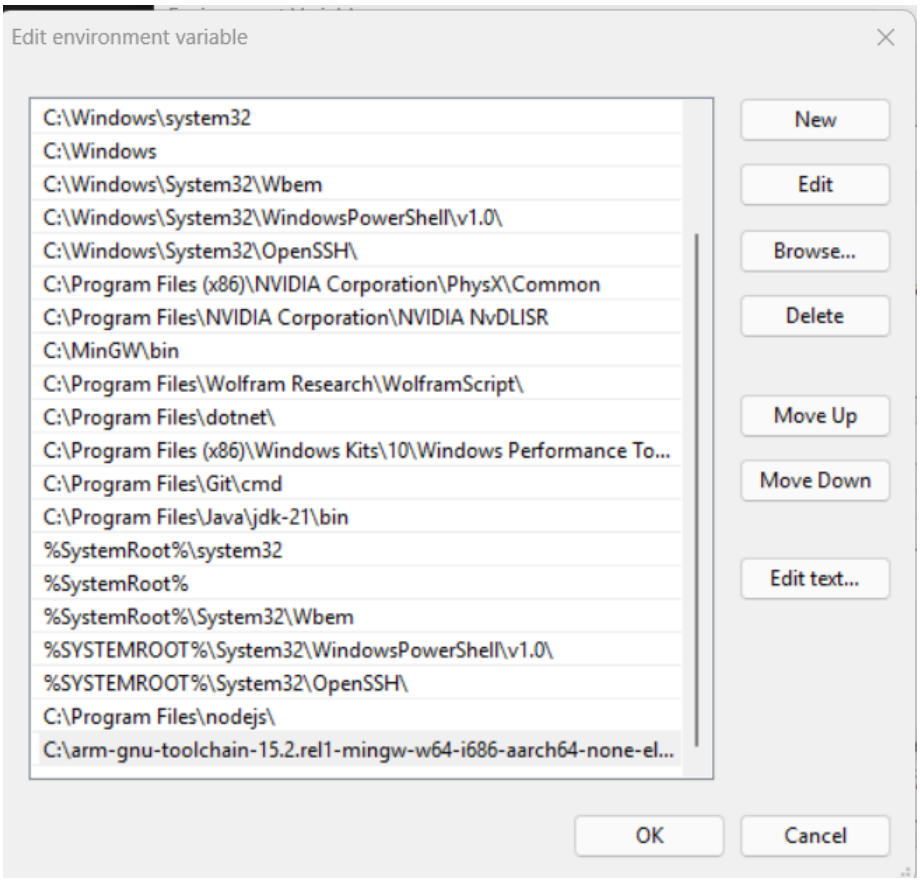
AArch64 bare-metal target (aarch64-none-elf)

- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.zip
- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.zip.asc
- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.zip.sha256asc
- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.msi
- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.msi.asc
- arm-gnu-toolchain-15.2.rel1-mingw-w64-x86_64-aarch64-none-elf.msi.sha256asc

Link Download: https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads

Step 2: Setup the Environment

Copy “C:\arm-toolkit\bin” to Environment Variables



Step 3: Use command to convert file

Open CMD in folder containing file.c and use this command:
`aarch64-none-elf-gcc -S -O0 -fno-asynchronous-unwind-tables test.c -o test.s`

- `-S`: Tells the compiler to generate only the assembly code file (`.s`) and not create a finished program.
- `-O0`: Tells the compiler to keep the code exactly as written. It does not try to shorten or rearrange the steps, which makes it much easier to compare the C code with the Assembly code.
- `-fno-asynchronous-unwind-tables`: Removes extra technical data (like `.cfi` tags) used for error tracking. This leaves only the main CPU instructions, making the output look "clean" and simple.